

Target Tracking Using Quadrotor

Akanksha Singal (2021008)

December 17, 2023

Abstract

The aim of any AI based decision-making application is to replicate human like intelligence. Here, the focus is on reducing the training data and trainable parameters by using compact continuous-time neural models and carefully constructed trajectories. The Liquid Neural model understands relevant features from visual data and focuses its attention on these features to build generalizable and long causal systems. This brain-inspired model coupled with deep learning architecture learns to map visual data to control commands for quadrotors.

1 Introduction

Human perception for tasks like target tracking is different as compared to the existing conventional deep learning methods. Human intelligence involves two key steps that are - identifying the target and focusing all of our attention on the target when moving towards it. In contrast to the conventional deep learning models, humans do not scan their entire environment and identify the target after each step. Attention is sufficient to move towards the target. The brain-inspired Neural Circuit Policies is a sparsely wired four-layer architecture built by Liquid Time Constant cells whose structural and semantical architecture is inspired by the nervous system of the *Caenorhabditis elegans* worm.

In case of existing neural models, the neurons in each layer follow the below mathematical equation to map each input number to an output number as shown in Figure 1. These networks are series of matrix operations on discrete layers. The input data times weight matrix added to the bias matrix gives a matrix of predicted values which in turn acts as input to the next hidden layer. Recurrent Neural Networks add additional matrices calculated in previous layers as input to the current layer to account for time dependent data.

Neural Ordinary Differential Equations on time series data improve predictions as compared to all variations of Recurrent Neural Networks. Instead of discrete layers, Neural ODEs, consider neural networks as continuous functions. Experiments show this results in faster, more accurate results for continuous time models (time series data), and incorporating irregularity in the data. Neural ODEs are continuous time network as the state of a neural ODE, $x(t)$, is given by the solution of the below equation with a neural network f parametrized by θ . Here, $x(t)$ is the hidden state, $I(t)$ is the input and t represents time.

$$\frac{dx(t)}{dt} = f(x(t), I(t), t, \theta) \quad (1)$$

Liquid Neural Networks model the neurons in a way to closely resemble a neuron inside a human or an animal's brain. This is done by making the equation of the neuron more complex to dynamically adapt during training based on the input it receives.

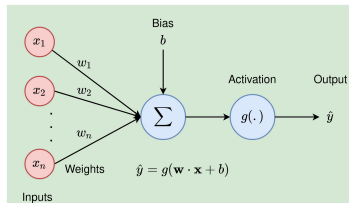


Figure 1: Conventional Neuron Equation (Source: Google)

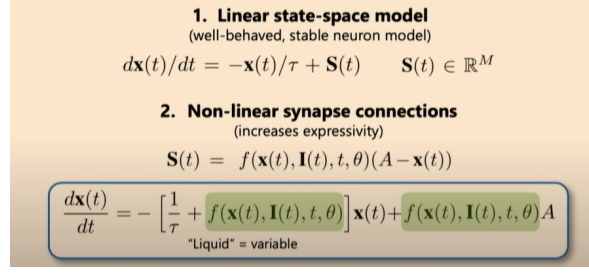


Figure 2: Neural Network modeled as ODE (Source: Google)

$$\frac{dx(t)}{dt} = -\left[\frac{1}{\tau} + f(x(t), I(t), t, \theta)\right]x(t) + f(x(t), I(t), t, \theta)A \quad (2)$$

$$\frac{dx(t)}{dt} = -\frac{x(t)}{\tau} - f(x(t), I(t), t, \theta)x(t) + f(x(t), I(t), t, \theta)A \quad (3)$$

The term $x(t)/\tau$ is a decay term that helps the system to reach an equilibrium state with a time-constant τ . It counteracts the effects of the inputs and parameters by pulling the state back towards zero or the equilibrium state when they are not present. Here, θ and A are parameters. Any ODE solver can be employed to get the original state of the neural network.

2 Methodology

2.1 Creating the environment

The Neural Circuit Policies architecture is a part of the custom keras-ncp library in python. It can be installed and set-up using its github repository. Here, the environment is setup using the docker image provided by the authors.

2.2 Problem Statement

For the purpose of this IP, the original problem statement has been broken down into smaller task. In this report we are training the model on the dataset where target object i.e. the 'rover' represented by the ArUco marker is static and the drone will reach a goal position and orientation with respect to the marker.

2.3 Understanding the Dataset

The dataset is present in the 'dataset' folder in the github repository. Inside this directory are multiple directories labeled 1 to 10. Each of these directories labeled 1 through 10 represent a unique trajectory followed by the drone to reach the goal position and orientation. There are RGB-images capture by the drone at regular timestamps and a csv file containing the corresponding 'Vx, Vy, Vz and Omega z' control inputs for each timestamp. The first row in every csv file are the labels 'Vx, Vy, Vz and Omega z'. The sequence length is unique to each trajectory. All images are resized to the dimensions (144, 256, 3) which are the height, width and channels (RGB) respectively while creating the tensorflow Dataset as the training data. Image augmentation techniques can also be employed on the dataset to increase the number of training samples and increase the adaptability of the model in unseen environments.

2.4 Model Architecture

The model is divided into two main parts - discrete part and the continuous part. The discrete part involves deep learning network for feature extraction from the images using Convolutional Neural Networks. As Figure 3 shows, the first layer is the input layer. The training data in the format of tensorflow Dataset is mapped to this input layer. The specifications of this input is represented as (batch [None], sequence length, image dimensions). Here batch size is given as 1, sequence length is

Model: "model"		
Layer (type)	Output Shape	Param #
input_1 (InputLayer)	[(None, 64, 144, 256, 3)]	0
rescaling (Rescaling)	(None, 64, 144, 256, 3)	0
time_distributed (TimeDistributed)	(None, 64, 144, 256, 3)	7
time_distributed_1 (TimeDistributed)	(None, 64, 70, 126, 24)	1824
time_distributed_2 (TimeDistributed)	(None, 64, 33, 61, 36)	21636
time_distributed_3 (TimeDistributed)	(None, 64, 15, 29, 48)	43248
time_distributed_4 (TimeDistributed)	(None, 64, 13, 27, 64)	27712
time_distributed_5 (TimeDistributed)	(None, 64, 6, 13, 16)	9232
time_distributed_6 (TimeDistributed)	(None, 64, 1248)	0
time_distributed_7 (TimeDistributed)	(None, 64, 128)	159872
time_distributed_8 (TimeDistributed)	(None, 64, 128)	0
rnn (RNN)	(None, 64, 4)	22398
Total params: 285,929		
Trainable params: 285,922		
Non-trainable params: 7		

Figure 3: Model Architecture

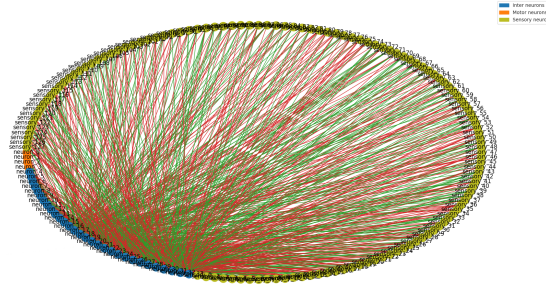


Figure 4: LTC wiring

1030 i.e. the number of images in the dataset, and image dimensions are (144, 256,3) which are the height, width and channels (RGB) respectively. Batching of the data set is done to increase efficiency and improve memory management of the model. When batching a new Dataset is created which takes the number of data samples from the training dataset equal to the batch size and adds that to the dataset. Hence the number of batch size of training samples now represent our new data point. This means that the corresponding matrix updations are done by performing the forward pass on these 'number of batch size' data points. The rescaling and normalization layers are added to ensure a standard range of the pixel values. Each two-dimensional convolution layer has a time distributed wrapper. This wrapper ensures that the same CNN is applied to each time step of the input to preserve the temporal structure of the sequence. The output of the 5 convolutional network is then flattened to 128 units/neurons and added on a dense layer. The dropout layer is added to prevent overfitting. The continuous part is the NCP model with the wiring as shown in Figure 4. The NCP model has only 3 types of neurons that are - Sensory, Inter and Motor neurons. The 4 motor neurons represent the 4 control values required as output of the model.

3 Results

The model was trained on a dataset with a single trajectory. We see in Figure 6 that the loss converges to 0.0028.

```
load dataset shape (TensorSpec(shape=(1030, 144, 256, 3), dtype=tf.uint8, name=None), TensorSpec(shape=(1030, 4), dtype=tf.float32, name=None))
```

Figure 5: Specifications of Input to the model

```

Epoch 21/30
1/1 [=====] - 155s 155s/step - loss: 0.0036 - mse: 0.0036
Epoch 22/30
1/1 [=====] - 11s 11s/step - loss: 0.0033 - mse: 0.0033
Epoch 23/30
1/1 [=====] - 11s 11s/step - loss: 0.0032 - mse: 0.0032
Epoch 24/30
1/1 [=====] - 11s 11s/step - loss: 0.0032 - mse: 0.0032
Epoch 25/30
1/1 [=====] - 11s 11s/step - loss: 0.0030 - mse: 0.0030
Epoch 26/30
1/1 [=====] - 10s 10s/step - loss: 0.0032 - mse: 0.0032
Epoch 27/30
1/1 [=====] - 10s 10s/step - loss: 0.0031 - mse: 0.0031
Epoch 28/30
1/1 [=====] - 10s 10s/step - loss: 0.0029 - mse: 0.0029
Epoch 29/30
1/1 [=====] - 10s 10s/step - loss: 0.0028 - mse: 0.0028
Epoch 30/30
1/1 [=====] - 11s 11s/step - loss: 0.0028 - mse: 0.0028

```

Figure 6: Training Output Screenshot

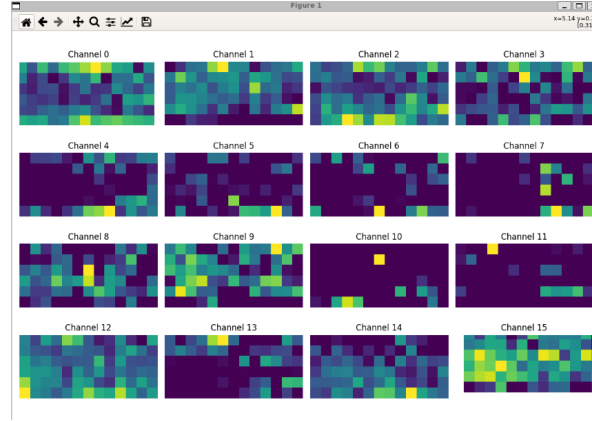


Figure 7: Highlighted features for the dataset Devens Chair

Figure 7 shows the feature highlight for the input image from the dataset used in the author’s paper.

Figure 8 show the input image to our model and Figure 9 show the features highlighted by the convolutional layers before flattening the output.

4 Conclusion

NCP is a compact and sparse network with only 22,398 parameters. The model can help realise end-to-end training from visual data to control outputs. The next steps involve incorporating variable length trajectories in the model and dynamic goal setting. One of the challenges include scenarios where the target is not in the drone’s field of view.

5 References

1. Robust flight navigation out of distribution with liquid neural networks by Makram Chahine, Ramin Hasani, Patrick Kao, Aaron Ray, Ryan Shubert, Mathias Lechner, Alexander Amini, Daniela Ru [Link to paper](#).
2. Liquid Time-Constant Networks by Ramin Hasani, Mathias Lechner, Alexander Amini, Daniela Rus, Radu Grosu [Link to paper](#).
3. Neural circuit policies enabling auditable autonomy [Link to paper](#).



Figure 8: Input Image to the Convolutional Layers

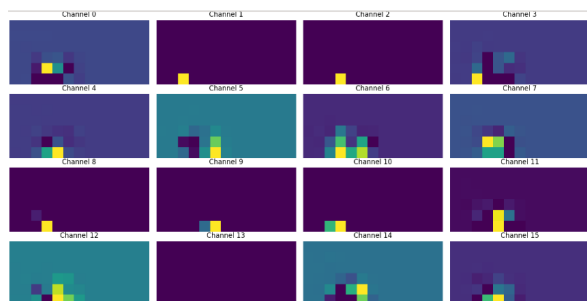


Figure 9: Highlighted features identified for the ArUco marker